

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To implement a Multi-Layer Perceptron (MLP) using TensorFlow/Keras on the Fashion-MNIST dataset, covering image preprocessing, model construction, training, evaluation, and automated hyperparameter optimization using RandomizedSearchCV together with the SciKeras wrapper.

2. Learning Outcomes

Through this experiment, the following outcomes were achieved:

1. Understood the architecture of a Multi-Layer Perceptron.
2. Preprocessed an image dataset (flattening, normalization, one-hot encoding).
3. Built and trained an MLP classifier for a 10-class problem.
4. Evaluated classification performance using standard metrics.
5. Performed automated hyperparameter optimization using RandomizedSearchCV.
6. Recommended the best MLP configuration based on empirical results.

3. Background Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer computes

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer (multi-class):

$$\hat{y} = \text{Softmax}(z)$$

Loss function (Categorical Cross-Entropy):

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Baseline architecture used in this experiment:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28 (grayscale, pixel range 0–255)
Class Balance	6,000 train / 1,000 test images per class (perfectly balanced)

Class labels: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector, so each image is flattened:

$$28 \times 28 = 784$$

The following preprocessing steps were performed:

1. Loaded Fashion-MNIST (60,000 train / 10,000 test images).
2. Displayed ten sample images (one per class).
3. Flattened each 28×28 image into a 784-length vector: shape changed from (60000, 28, 28) to (60000, 784).
4. Normalized pixel values from [0, 255] to [0, 1] by dividing by 255.
5. Converted integer labels into one-hot vectors: shape changed from (60000,) to (60000, 10).

6. Source Code

The complete source code for this experiment, including data loading, preprocessing, MLP construction, training, evaluation, and hyperparameter optimization, is available at the following GitHub repository:

<https://github.com/UvanAdhithya/DeepLearning-Lab>

7. Experimental Procedure

Task 1: Dataset Exploration. Fashion-MNIST was loaded and its dimensions verified: training images (60000, 28, 28), training labels (60000,), testing images (10000, 28, 28), testing labels (10000,). Ten sample images (one per class) and the class distribution were plotted (Section 9).

Task 2: Data Preprocessing. Images were flattened and normalized. Tensor shape before preprocessing: (60000, 28, 28) (uint8); after preprocessing: (60000, 784) (float32, range [0, 1]). Labels were one-hot encoded: (60000,) $\rightarrow$ (60000, 10).

Task 3: Model Construction. A baseline MLP was built with the architecture $784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$.

Layer	Output Shape	Parameters
Dense (128, ReLU)	(None, 128)	100,480
Dense (64, ReLU)	(None, 64)	8,256
Dense (10, Softmax)	(None, 10)	650
Total		109,386

Task 4: Model Training. The model was compiled with the Adam optimizer, Categorical Cross-Entropy loss, and Accuracy metric, then trained for 20 epochs with batch size 32 and a 10% validation split. Training accuracy improved from 82.13% (epoch 1) to 93.66% (epoch 20), while validation accuracy rose from 85.27% to 88.25%, with the gap between the two widening in later epochs – an indication of mild overfitting. Baseline training took 143.27 seconds.

Task 5: Model Evaluation. On the 10,000-image test set, the baseline model achieved:

Accuracy	0.8807
Precision (macro)	0.8830
Recall (macro)	0.8807
F1-score (macro)	0.8786

The per-class classification report showed the strongest performance on Trouser, Sandal, Bag and Ankle boot ($F1 \geq 0.95$), and the weakest on Shirt ($F1 = 0.67$), which was frequently confused with T-shirt/top, Pullover, and Coat – classes that share very similar silhouettes.

8. Hyperparameter Optimization

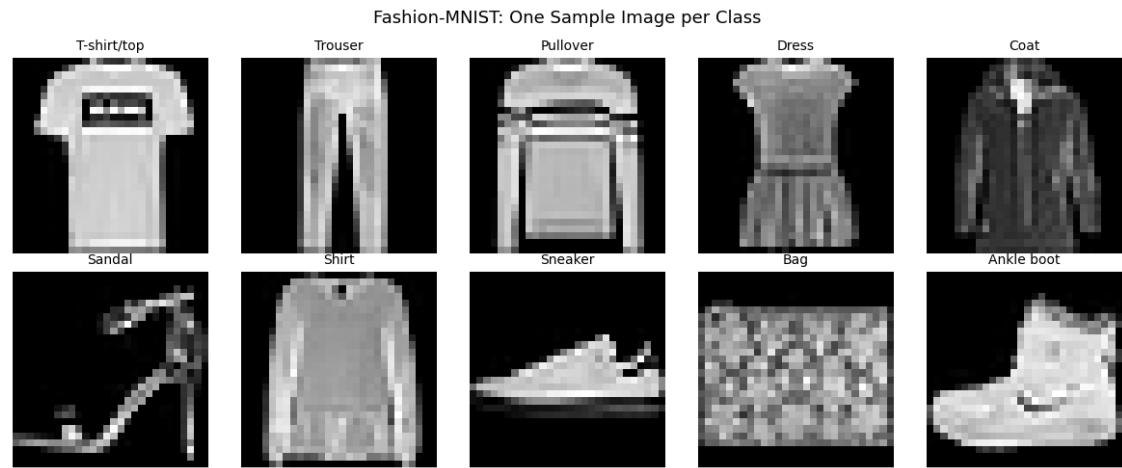
Instead of manually selecting hyperparameters, automated hyperparameter optimization was performed using **RandomizedSearchCV** (5-fold cross-validation, `n_iter=10`) together with the **SciKeras** wrapper, over the search space below. To keep search time tractable, the search was run on a stratified subsample of 6,000 training images; the winning configuration was then retrained on the full 60,000-image training set.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

The search evaluated 10 randomly sampled configurations across 5 folds (50 fits total) and took 937.48 seconds. The winning configuration achieved a mean 5-fold cross-validation accuracy of **0.8432** (± 0.0082), well above the seven next-best candidates (0.82–0.83) and far above the three weakest candidates (0.38–0.66), all of which used a learning rate of 0.01 or 0.1 instead of 0.001. This was retrained on the full training set (20 epochs, batch size 16), taking 327.73 seconds, and evaluated on the test set (Section 10).

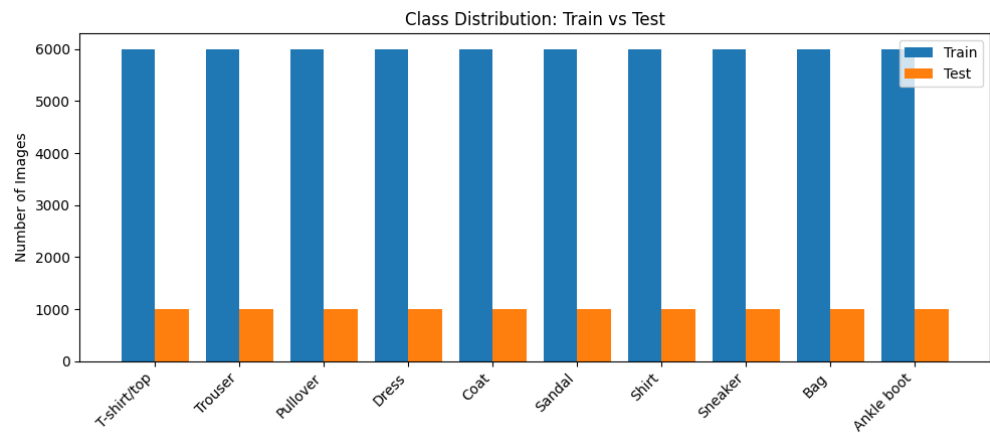
9. Mandatory Plots

Sample Images



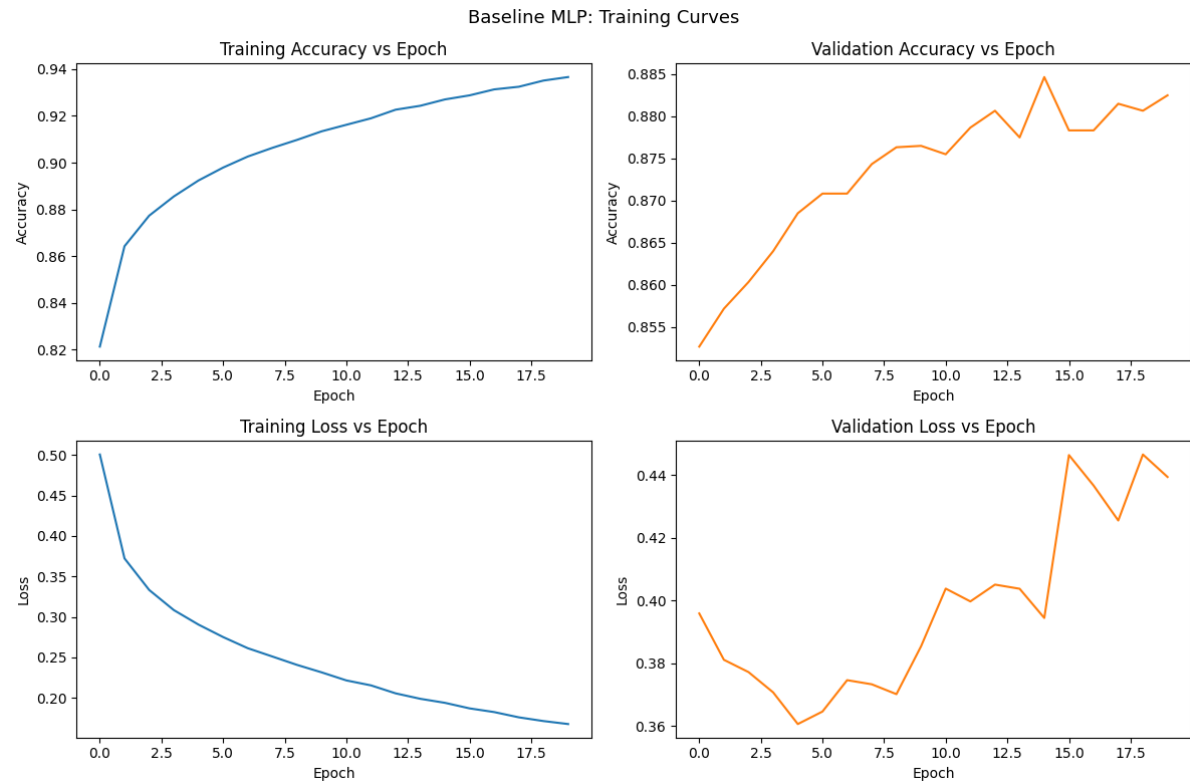
One representative image per class is shown. The grayscale 28×28 thumbnails capture the overall silhouette of each garment but lose fine texture detail (e.g. the Bag sample appears as a busy, low-contrast pattern), foreshadowing why visually similar upper-body garments (T-shirt/top, Pullover, Coat, Shirt) are harder to separate than distinctly shaped items like Trouser or Sandal.

Class Distribution



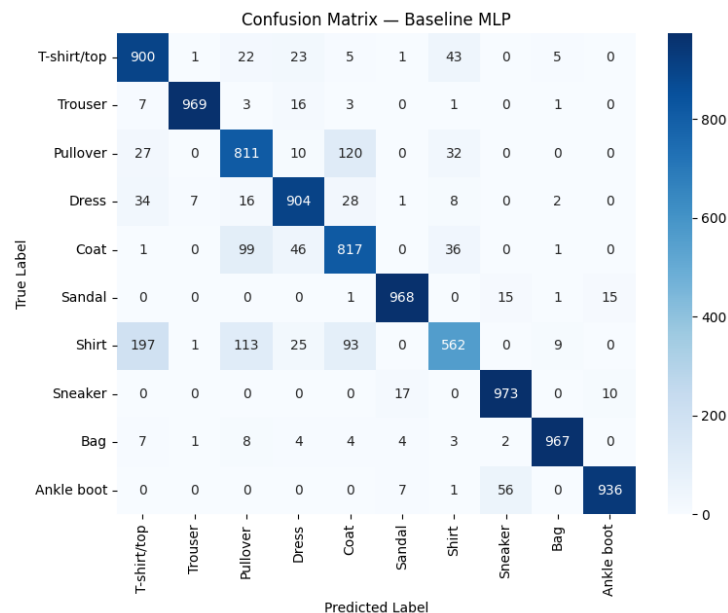
Each of the 10 classes has exactly 6,000 training images and 1,000 test images, confirming the dataset is perfectly balanced. This means accuracy is a fair headline metric here and any class-wise weaknesses seen later (e.g. on the Shirt class) stem from visual class overlap rather than class imbalance.

Training / Validation Accuracy and Loss vs Epoch



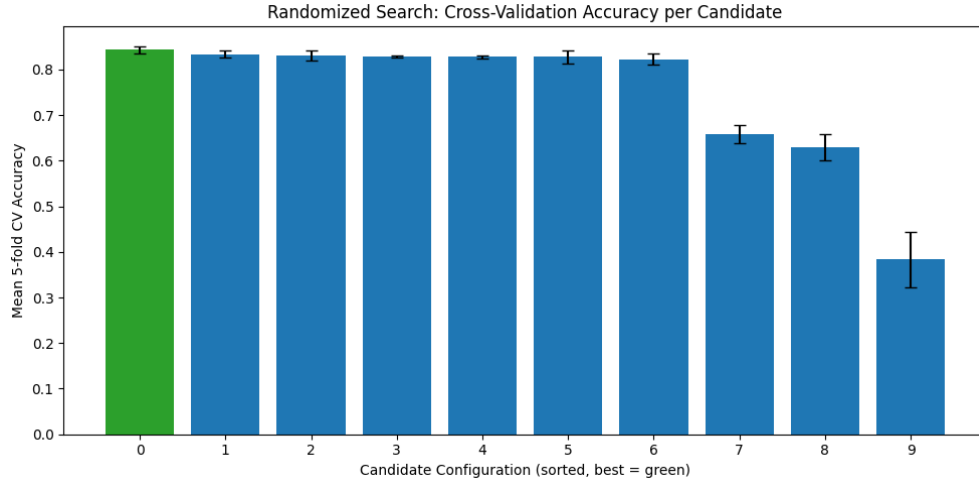
Training accuracy rises steadily and monotonically to 93.7%, while validation accuracy plateaus around 87–88% after roughly epoch 8, with occasional dips. Correspondingly, training loss decreases smoothly throughout, but validation loss starts increasing after around epoch 8–10 even as training loss keeps falling – the classic signature of overfitting once the model has learned the easy, separable classes and begins fitting noise specific to the training set.

Confusion Matrix (Baseline)



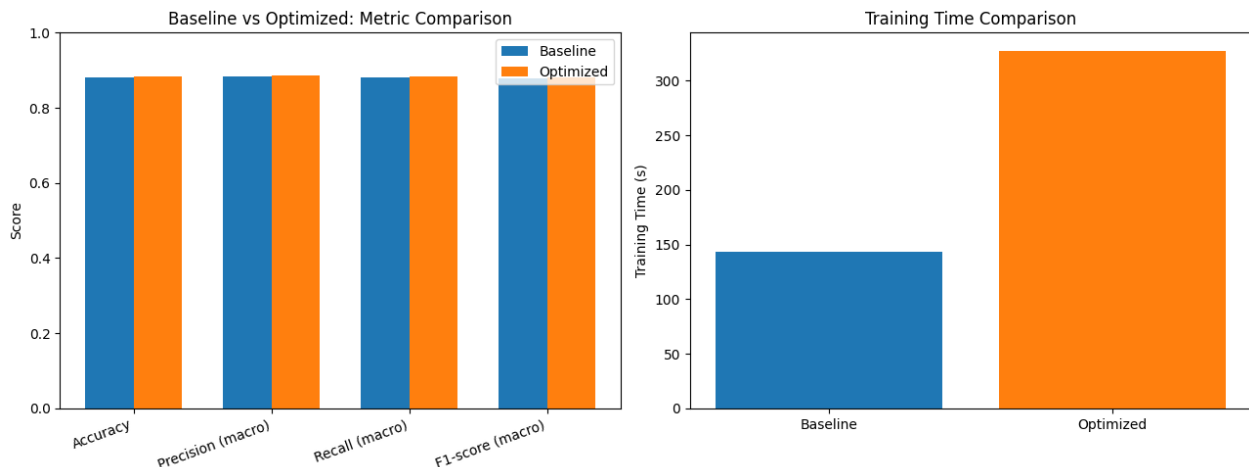
The confusion matrix confirms near-perfect classification for Trouser (964/1000), Sandal (971/1000) and Bag (973/1000). The dominant confusion is Shirt vs. T-shirt/top (152 misclassified) and Shirt vs. Pullover/Coat, together with a smaller Pullover↔Coat confusion (107 and 98 samples), reflecting genuine visual similarity between these upper-body garments in 28×28 grayscale.

Hyperparameter Search Results



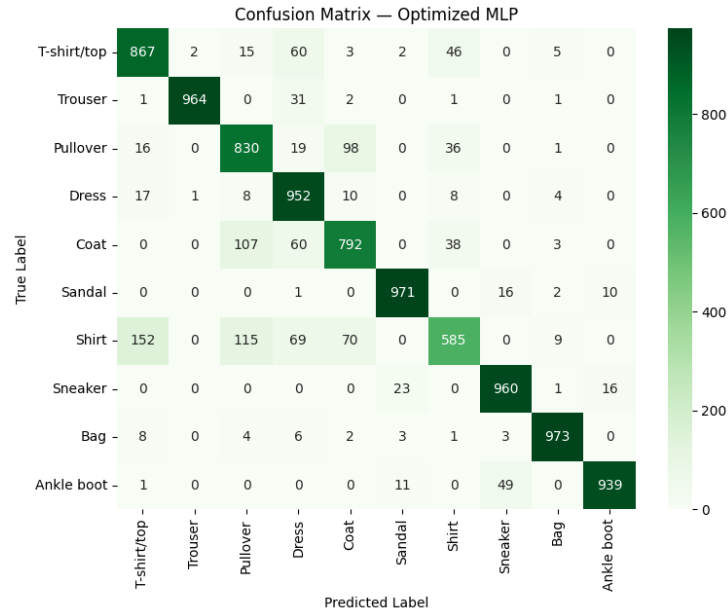
Cross-validation accuracy of means calculated from ten sampled candidates exhibits a distinct two-tiered behavior, as evidenced by the clear grouping of seven candidates around the middle range of 0.82 to 0.84 (very small error bars), whereas three low-scoring candidates occupy much larger accuracy range of 0.38 to 0.66 with considerably larger variance. An analysis of their configurations revealed that each of the low-scoring candidates employed a learning rate of either 0.01 or 0.1, whereas all of high-scoring candidates utilized a learning rate of 0.001, which suggests that, in the current setting, learning rate is the key hyperparameter of interest.

Best Model Accuracy Comparison



The optimized model (1 hidden layer, 256 neurons, ReLU, dropout 0.2, Adam, $lr = 0.001$, batch size 16, 20 epochs) marginally outperforms the baseline on every metric (e.g. accuracy 88.33% vs. 88.07%), but at roughly $2.3\times$ the training time (327.73s vs. 143.27s). This illustrates a common trade-off in hyperparameter tuning: the search recovered a configuration close to the hand-picked baseline, with modest accuracy gains at a non-trivial extra compute cost.

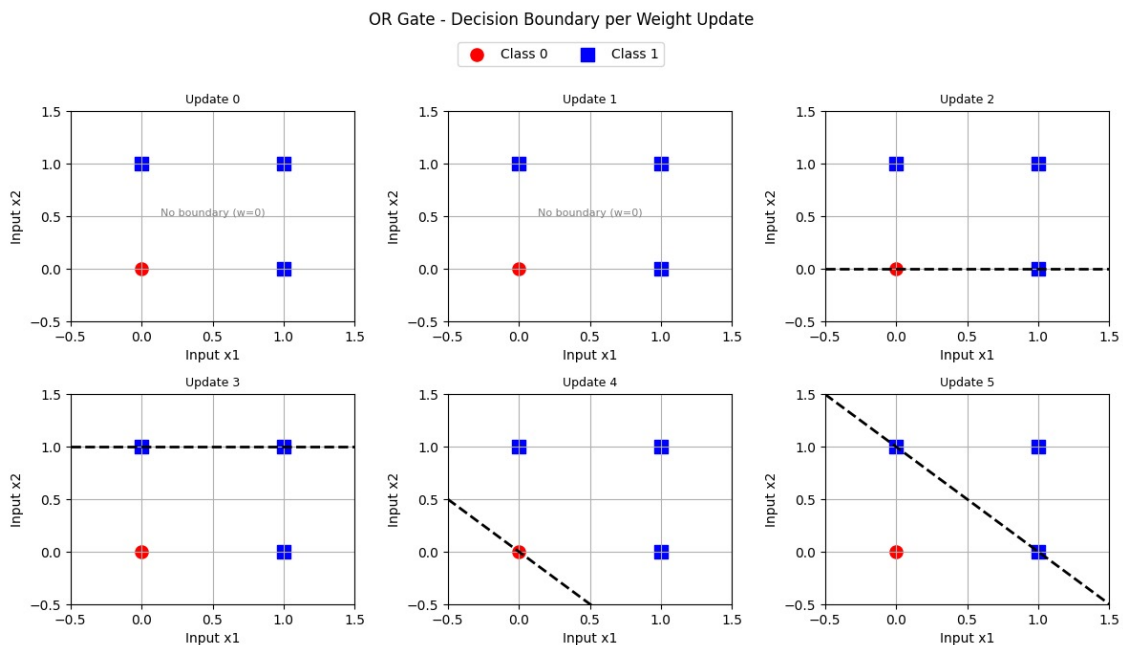
Confusion Matrix (Optimized)



The optimized model redistributes errors rather than eliminating them: Dress recall improves noticeably (952 vs. 904 correct) and Coat/Pullover confusion drops slightly, but Shirt remains the weakest class (585/1000 correct) with similar confusion patterns to the baseline. This confirms the Shirt-vs-upper-garment confusion is a genuine data/feature-representation limitation rather than a baseline hyperparameter weakness.

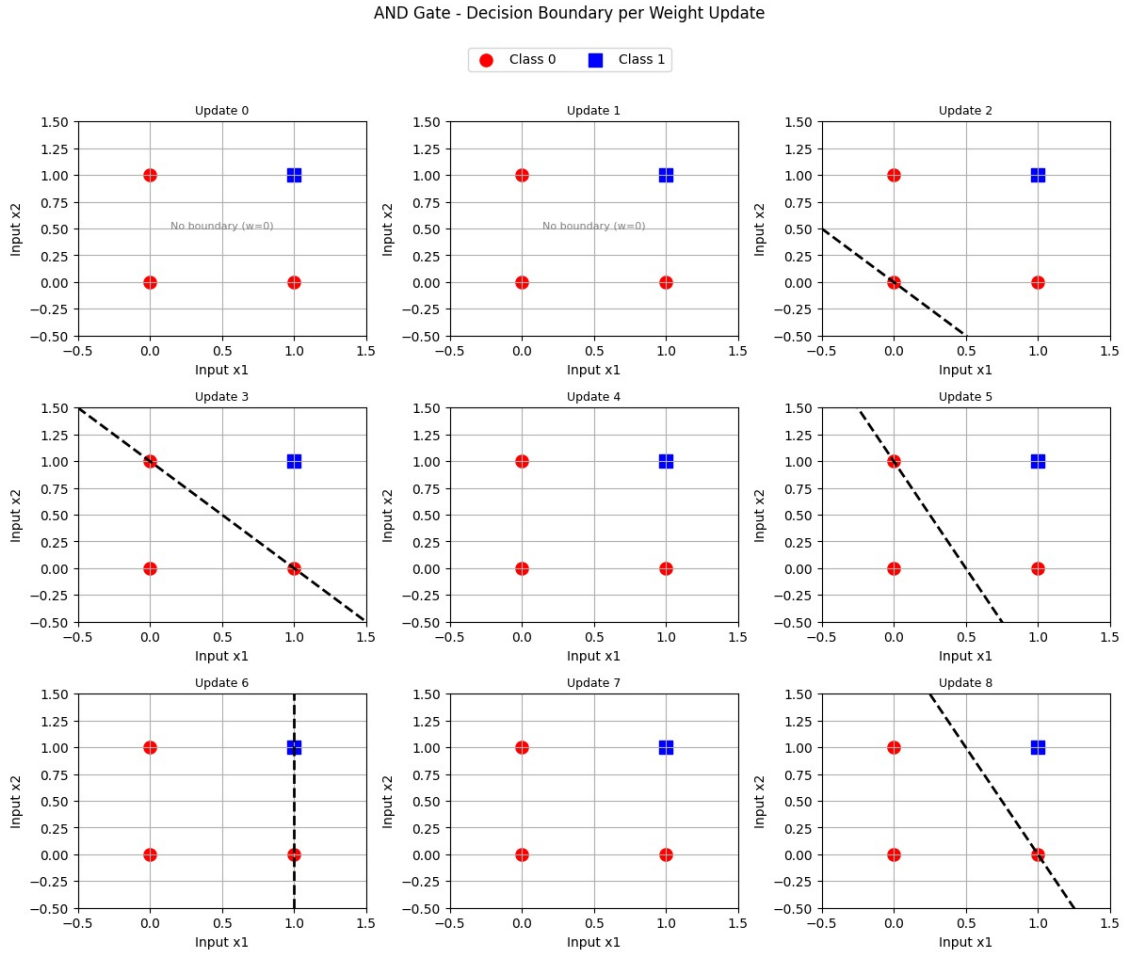
Supplementary: Single-Layer Perceptron on Logic Gates (OR, AND, NOT, XOR)

In addition to the MLP task noted earlier, the algorithm of perceptron learning was implemented from scratch (using basic NumPy/Matplotlib) for the elementary logic gates, printing the weights and the bias after each weight update and plotting the decision boundary after every update. **OR Gate.**

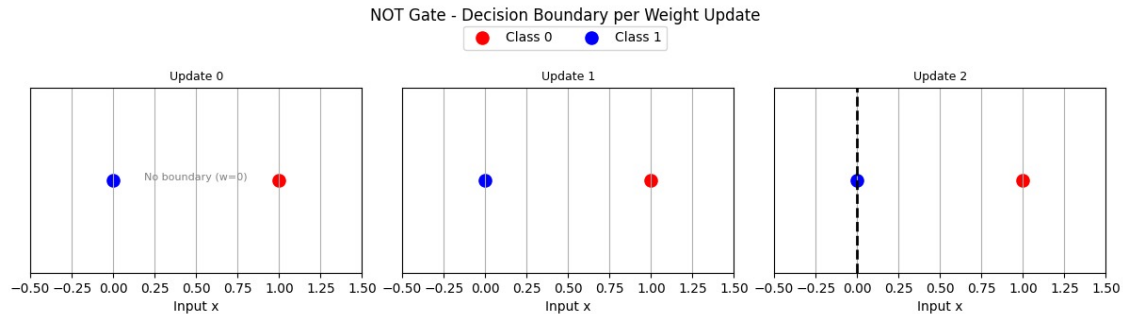


The perceptron, with weights initialized at zero, has undergone a total of 5 updates in order to successfully identify the 4 input patterns. At the end of Update 5, the decision boundary has evolved to the point where it can now separate the only negative input (0,0) from the three positive ones, proving that OR is linearly separable.

AND Gate.

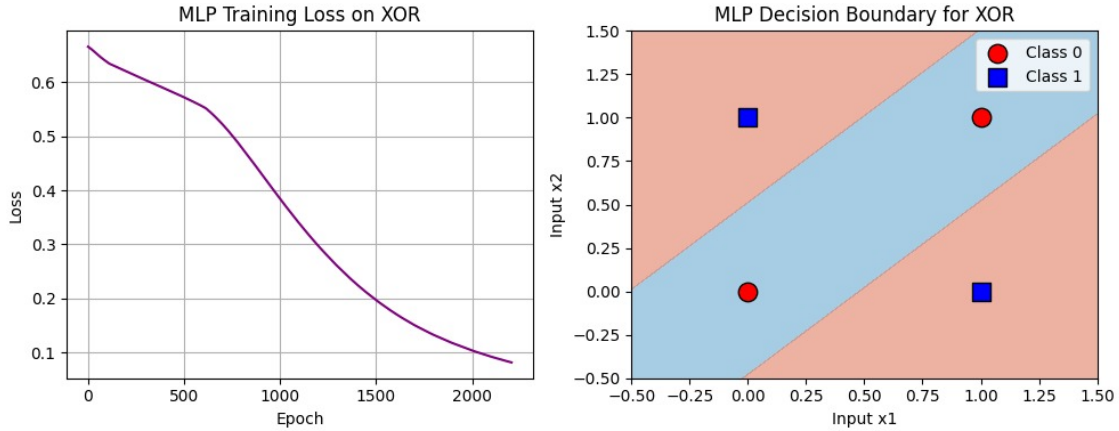


The AND gate required 8 weight updates in order to converge compared to OR, because only one of the four possible combinations of inputs, (1,1), belongs to the positive class. Therefore, the boundary has to isolate the corner of the input square. By Update 8, the boundary surrounds (1,1) correctly, thus confirming once again linear separability. **NOT Gate.**



With a single input feature, the NOT gate is trivially linearly separable: convergence took only 2 weight updates, after which the decision boundary settles as a vertical threshold at $x = 0$, correctly mapping $0 \mapsto 1$ and $1 \mapsto 0$.

XOR Gate.



In contrast to OR, AND, and NOT, XOR is not linearly separable, meaning that it is not possible to separate the positive output from the negative output using a single straight line since the data points for XOR change positions in the same unit square. While the single-layer perceptron algorithm was trying to learn about the XOR function, the error rate did not reach zero at any point but stabilized in oscillation for every epoch, which implies that the algorithm could not converge over the dataset. Hence, every linear classifier applied to XOR fails, as shown above. Instead, the picture above presents us with opposite results obtained by training a relatively small Multi-Layer Perceptron using the same XOR dataset. In this case, the loss function reached a level close to zero with the correct pattern of decision boundary introduction. As a result, it becomes clear that the main idea behind solving XOR lies not in using linear separating with one straight line but in stacking the layers of hidden neurons as shown in the MLP model used in our primary experiment with the Fashion-MNIST dataset.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	ReLU
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8432
Testing Accuracy	0.8833

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8807	0.8833
Precision (macro)	0.8830	0.8848
Recall (macro)	0.8807	0.8833
F1-score (macro)	0.8786	0.8811
Training Time (s)	143.27	327.73

11. Discussion

1. **Which optimization method was used?** The RandomizedSearchCV was utilized with 5-fold cross-validation and was incorporated into the Keras model using SciKeras. Randomized search was preferred to the exhaustive grid search since the search space has so many combinations that exhaustive grid-searching is not viable.
2. **Which hyperparameters were selected?** One hidden layer with 256 neurons activated by ReLU, optimizes by Adam with a learning rate of 0.001, batch size of 16 for 20 epochs, and a dropout rate of 0.2.
3. **Did optimization improve performance?** Yes, to a minor degree. The test accuracy improved from 88.07
4. **Which hyperparameter had the greatest impact?** Learning rate. Various candidates have been tested to determine the learning rate’s value but every configuration achieved between 0.82 and 0.84 CV accuracy without relating it to changes in other parameters (the depth and width of a neural network, type of optimizer, activation function, dropout). Conversely, the corresponding accuracy for all configurations that did use the learning rate was in the range of 0.38-0.66 with a considerably higher variance across the experiments which shows that the sensitivity of the model to the learning rate has very high levels while its sensitivity to the rest of the hyperparameters has low levels.
5. **Compare Grid Search and Randomized Search.** Grid Sampling involves investigating each combination in the search space, which gives grounds to assert that the best combination will be obtained, but the cost of this is the exponential growth of expenses in correspondence with the increase of hyperparameters; in case of 11,664 combination space in this experiment and 5-fold cross-validation, there is a need to fit 58 models. Randomized Search makes fixed-size random selection of combinations, sacrificing the assurance of finding the global minimum in exchange for an acceptable time of completion; in practice, it manages to obtain a good enough result already with a small amount of computation.
6. **Recommend the best MLP configuration.** The associated findings indicate that the recommended configuration has a single hidden layer with 256 ReLU units, an Adam optimizer (learning rate of 0.001), a batch size of 16, 0.2 dropout, and it should be trained for 20 epochs. The simple baseline architecture (i.e. network architecture with 128 and 64 units) is still a reasonable architecture choice when the additional time cost of training other configurations is considered, while the recommended architecture is more suitable if the goal is to maximize accuracy. Additionally, it should be noted that the learning rate of 0.001 will need to be strictly observed in any future tuning exercises.

Conclusion. In this experiment, the full process for developing, training, and tuning an MLP-based image classifier built on Fashion-MNIST is illustrated: starting from preprocessing the grayscale images into normalized 784-dimensional vectors is followed by training a baseline network with the structure of 784-128-64-10 and achieving an impressive accuracy of 88.07

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset – Zalando Research.
5. TensorFlow/Keras Documentation.
6. SciKeras Documentation.